Carnegie Mellon University
Master of
Software Engineering

17-647 Assignment A2: BFF, JWT & Multiple Services per SRP

Problem Description

In this project assignment, you will apply the Backend For Frontends (BFF) pattern to your system, and add authentication control to your API with JWT. You will create an increment of the e-commerce system that you started in A1. The requirements and context in which this system will be used will evolve as the course progresses, so be sure to check the specifications provided in previous assignments and make sure your system complies.

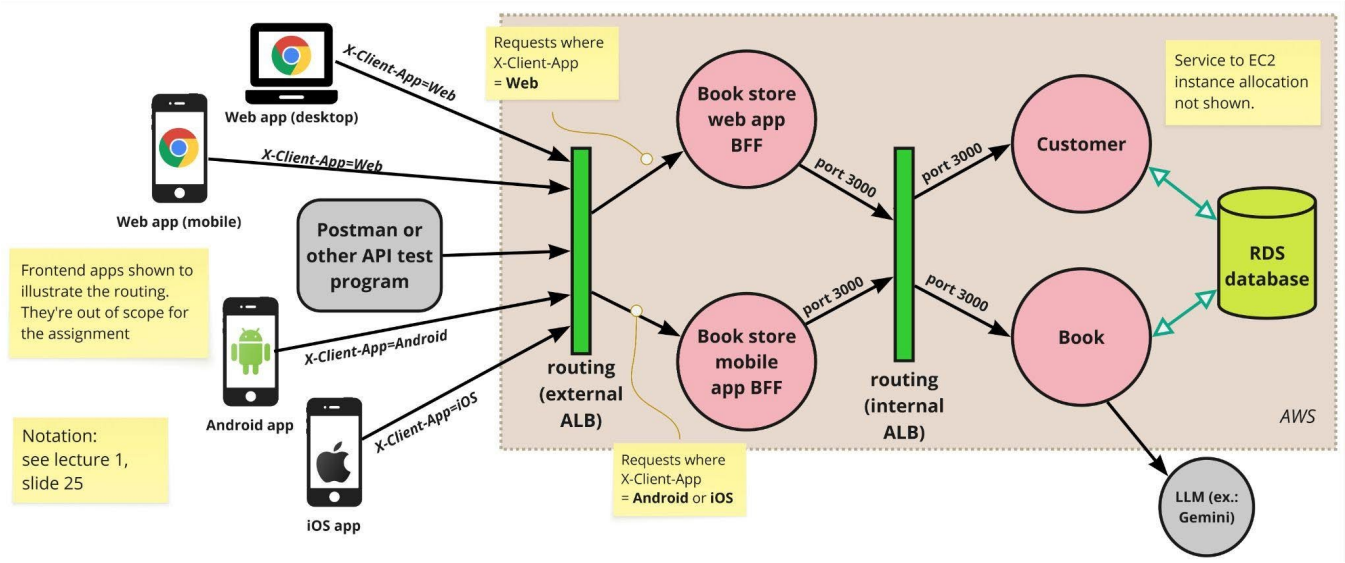
Learning Objectives

By the end of this assignment, you should be able to configure the needed infrastructure, develop a simple application, deploy your application, and execute your system, and (hopefully) be able to pass test cases generated by an automated testing tool. These technologies include:

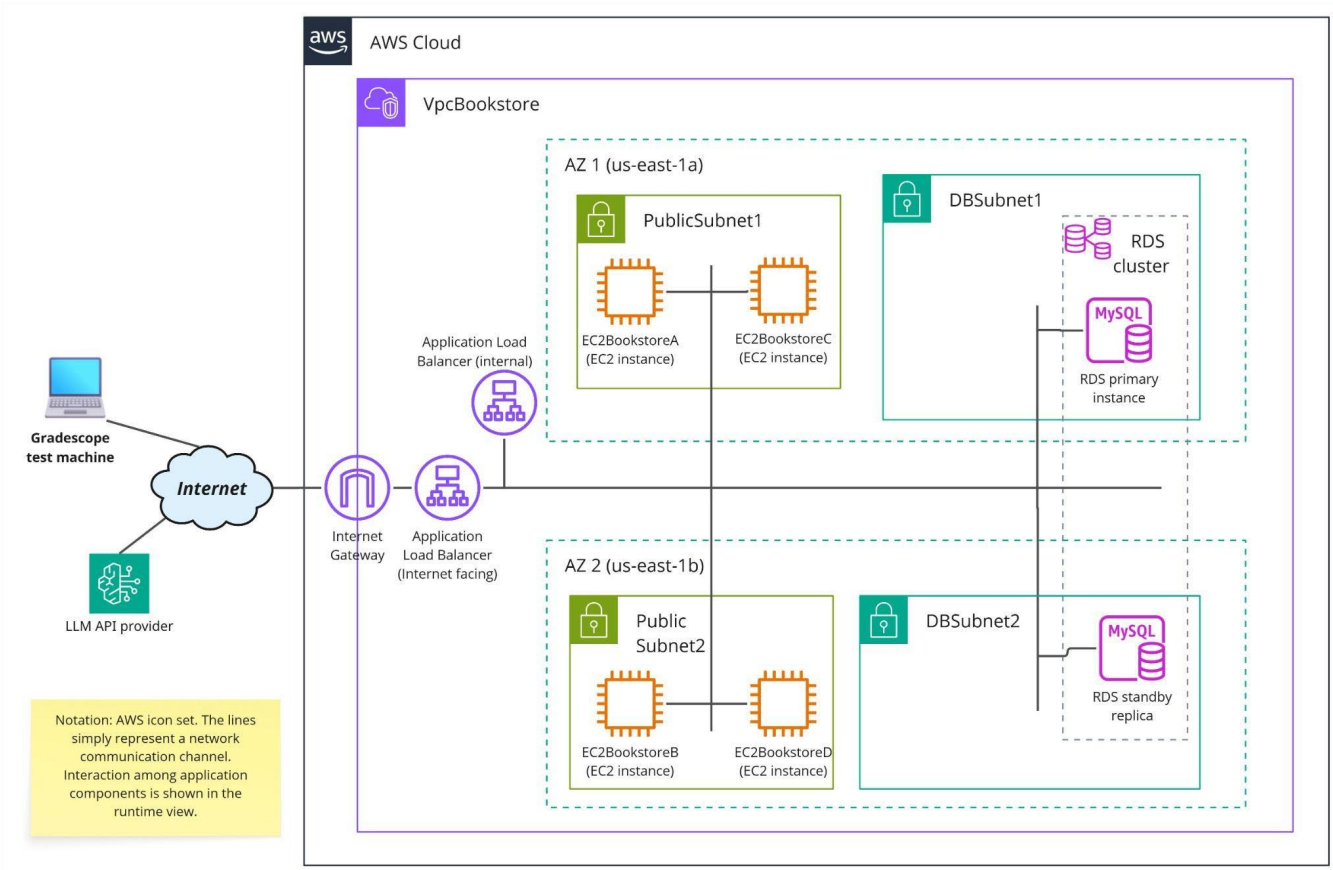
- AWS services: EC2, VPC, CloudFormation (CF)
- AWS RDS
- Load balancer and passive redundancy
- Postman (or similar API testing tool)
- Docker
- Python with Flask, Node JS and Express or other language/framework that you choose to create your services
- Service-to-service HTTP interaction
- JWT (added in A2)
- BFF pattern (added in A2)
- Single Responsibility Principle, moving from a monolithic service into microservices (added in A2)

Architecture overview

The diagram below shows the **runtime architecture** perspective that you should implement for this assignment.



This other diagram shows the **AWS deployment** perspective. This diagram uses the standard AWS notation for AWS infrastructure design diagrams.



The allocation of services to EC2 instances should be as indicated in the table below. Observe the ports each service shall listen to, as two microservices cannot listen to the same port on the same machine. The CF-A2-cmu.yml template has target groups corresponding to these subsets of instances.

Service	Port	EC2 instances it should be deployed to
Book store web app BFF	80	`EC2BookstoreA, EC2BookstoreB`
Book store mobile app BFF	80	`EC2BookstoreC, EC2BookstoreD`
Customer service	3000	`EC2BookstoreA, EC2BookstoreD`
Book service	3000	`EC2BookstoreB, EC2BookstoreC`

Tasks

In this assignment, you will separate your system into **multiple microservices**. This is something you may want to do in practice for a number of reasons, including the ability to scale the capability related to books independently from the

capability related to customers. For example, in a real scenario you would want to do that if book endpoints are called far more often than customer endpoints.
Also, Books and Customers represent distinct functional contexts. Per the **Single Responsibility Principle** (SRP), they should be implemented separately. Single responsibility is letter 'S' in IDEALS. In our particular case, we want *books* responsibilities and *customers* responsibilities in separate microservices

In addition, you will implement a **BFF** for two different types of clients. BFF services can provide various capabilities. In our case, they will identify the type of client that is making the request (web app or mobile iOS/Android) and configure the response accordingly.

The HTTP request to the BFF services must contain the `X-Client-Type` custom header, which specifies the client component. Your solution should route requests based on the header value as indicated in the table below.

Value of `X-Client-Type`	**What to do**
`Web`	Route to the Book store **web app BFF** service
`iOS`	Route to the Book store **mobile app BFF** service
`Android`	Route to the Book store **mobile app BFF** service
header not present	Reply with a **400 error**

The routing shall be performed by the ALB. The provided CF template (CF-A2-cmu.yml in this github repo) might be incomplete. Read through the CF template to identify where the routing based on the `X-Client-Type` header is performed. **Complement the template for routing rules that might be missing.**

The **client type (mobile or web app) will dictate the expected response** as explained further below.

In addition to the `X-Client-Type` header, all requests to the BFF services must contain a **valid JWT token** (encoded with the HS256 algorithm). For simplicity, we are not going to implement the process to *obtain* a valid token. This process occurs through the interaction of the frontend component with an authentication broker service. Instead we will assume that the end-user is already signed in and the client application has a valid JWT token.
The JWT token goes in the `Authorization` header. This is a standard HTTP header. Here's an example of value for this header (Bearer <token>):


```
Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJzdGFybG9yZCIsInJvbGVzIjoicGlsb3QsaGVybyxkaiIsImlzcyI6IlMuSC5JLkUuTC5E
IiwiZXhwIjoxNzQwOTA5ODE2LCJ1c3JuIjoiUGV0ZXIgUXVpbGwiLCJpYXQiOjE3NDA5MDYyMTZ9.2R7inlfYKGURQl8bo3-
AYxhLQKXa0KDx37Zvdy95l3Y
```


You can decode this string at jwt.io to see the actual JSON structure that corresponds to that JWT token. You can also use the JWT Encoder option to create your own JWT tokens for tests.

Your BFF services shall decode and validate the JWT token. There are different libraries to decode the JWT token. For this assignment you just need to decode the payload (2nd part of the token) to obtain a JSON string and then parse the JSON string into a JSON object. To simulate token validation, your BFF service shall verify these conditions:

- Token payload contains a "sub" claim[1] and its value is one of "starlord", "gamora", "drax", "rocket", or "groot".
- Token payload contains an "exp" claim in numeric date format and the date is in the future, that is, the token is not expired.
- Token payload contains an "iss" claim and its value is "cmu.edu".

If the JWT token is not present, not valid, or doesn't satisfy the conditions above, the service must respond with **status code 401**.

The services in A2 will expose the same endpoints as specified in A1 (including the `GET /status` endpoint), with the following changes:

- The Customers service will expose `/customers` endpoints; the Books service will expose `/books` endpoints.
- BFF services will expose both `/customers` and `/books` endpoints.
- BFF services will use **port 80** whereas the backend services will use **port 3000**.
- Both BFF services should be visible at the same IP address (same domain name), which is typically the ALB address.
- The `X-Client-Type` header is mandatory. If not present, the caller shall receive a 400 error.
- BFF services require the `Authorization` header. If it's not present, the client shall receive a 401 error.
- The `Authorization` header shall have a valid JWT token that satisfies the conditions indicated above. If the token is not valid, the service shall issue a 401 error.
- The `GET /books/isbn/{ISBN}` endpoint (as well as `GET /books/{ISBN}`) of the **Book store mobile app BFF** service shall replace in the response body the word "non-fiction" with the numeric value 3. (We can imagine the native mobile apps using a lookup table for genres.)
- The `GET /customers/{id}` and `GET /customers?userId={userId}` endpoints of the **Book store mobile app BFF** service will remove the following attributes in their responses: `"address"`, `"address2"`, `"city"`, `"state"`, and `"zipcode"`.

Different from what you created in A1, this assignment requires that you create:

- 4 separate services deployed as 4 different microservices, therefore
- 4 separate Dockerfiles

You are expected to use good programming practices with proper error handling.

You will also need to be sure to comply *exactly* with the specification provided. Any deviation will result in a failed test case even if the functionality is implemented correctly.

Technologies and Resources

In this assignment, you should also configure the AWS infrastructure and deploy your application using our Learner Lab course in the AWS Academy platform. To set up the infrastructure, you should download and use the CF-A2-cmu.yml template from this github repo.

Like before, you will be using Docker, MySQL, and AWS for this project. See assignment A1 for some introductory links if needed. Use Piazza or TA office hours if you need assistance with the assignment.

[1] In the JWT RFC, a *claim* is a name/value pair. For example, `"sub": "starlord"` is a claim that the subject (aka principal, i.e., the user ID) is "starlord".

Carnegie Mellon University
Master of
Software Engineering

A2 hints and tips

- The BFFs need to call the backend services via port 3000. As the runtime view of the architecture shows, this call goes through another ALB (configured for internal traffic). Therefore, when we start the BFFs we need to pass on the "BASE URL" of the backend services. What should it be? 🤔 You guessed it right, it's the DNS of the internal ALB.

- An easy way to pass on the BASE URL to your BFF is via the `-e` option in `docker run`. Similar to what you probably do for DB credentials. For example, the docker run may have an argument like this:
  `-e "URL_BASE_BACKEND_SERVICES=http://internal-InternalALB-822474286.us-east-1.elb.amazonaws.com:3000"`

- In A2 there are 4 different microservices at play with 2 replicas of each distributed across 4 EC2 instances. Sometimes we run all these instances and want to know if they're being called. A poor ~~man's~~ person's solution is to add "print" statements to your services. Then you can SSH to your EC2 instances and run the images with the `2>&1` argument (and without `-d`). All stdout and stderr the container prints will show on that shell window. Example:
  `docker run -p 3000:8080 -e pmerson/hello-boot-kotlin:latest 2>&1`

Submission

The same instructions for Gradescope submission used in A1 are going to be used for A2. There are 4 microservices in this assignment. They should be clearly separated in subdirectories.

Grading

The grading for this project will be:
- The requirements were correctly implemented.
- The system is able to handle correct and invalid input and return the correct HTTP code in the response.
- The architecture of the system matches the instructions provided.